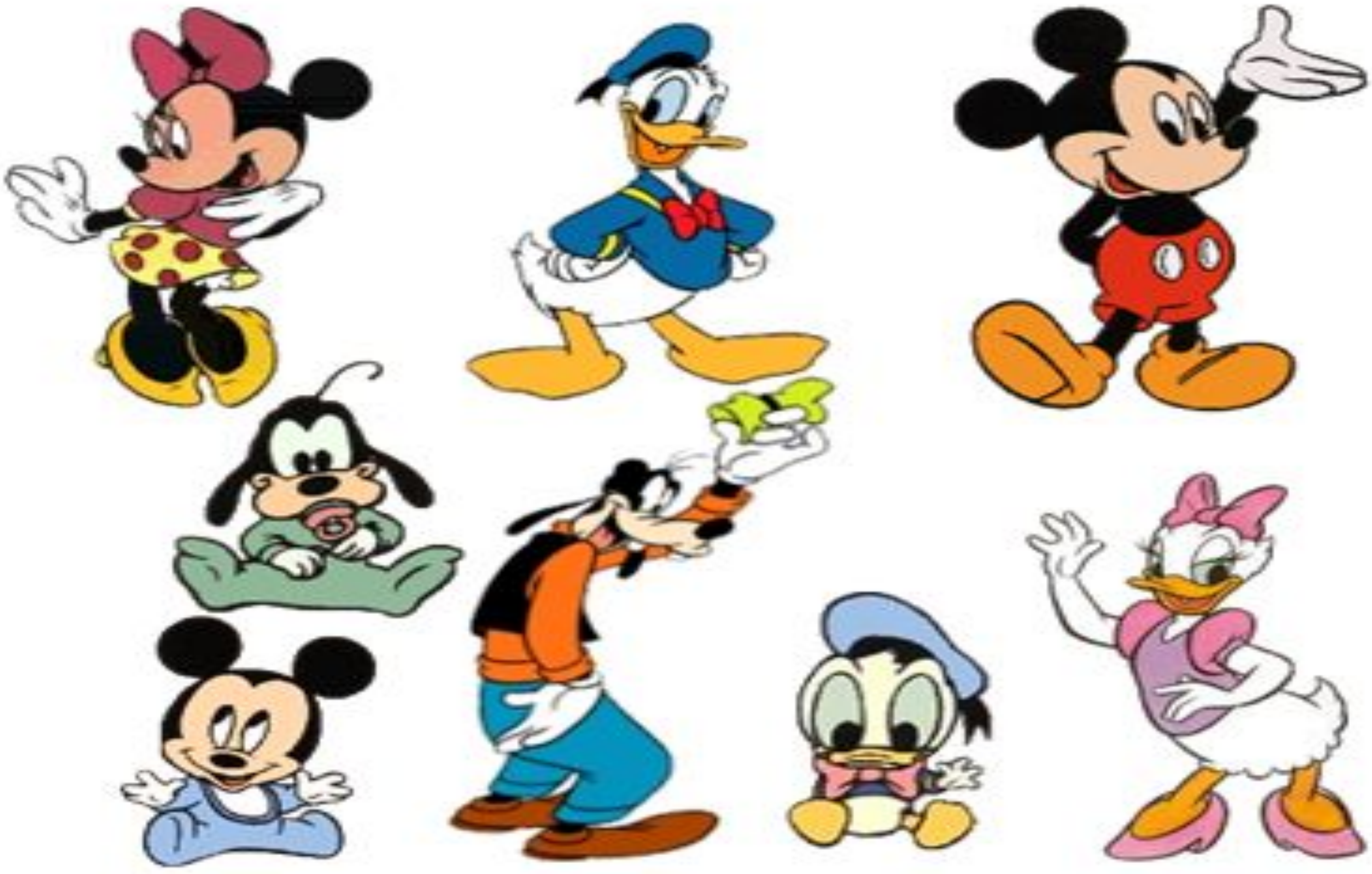
What is Cluster Analysis?

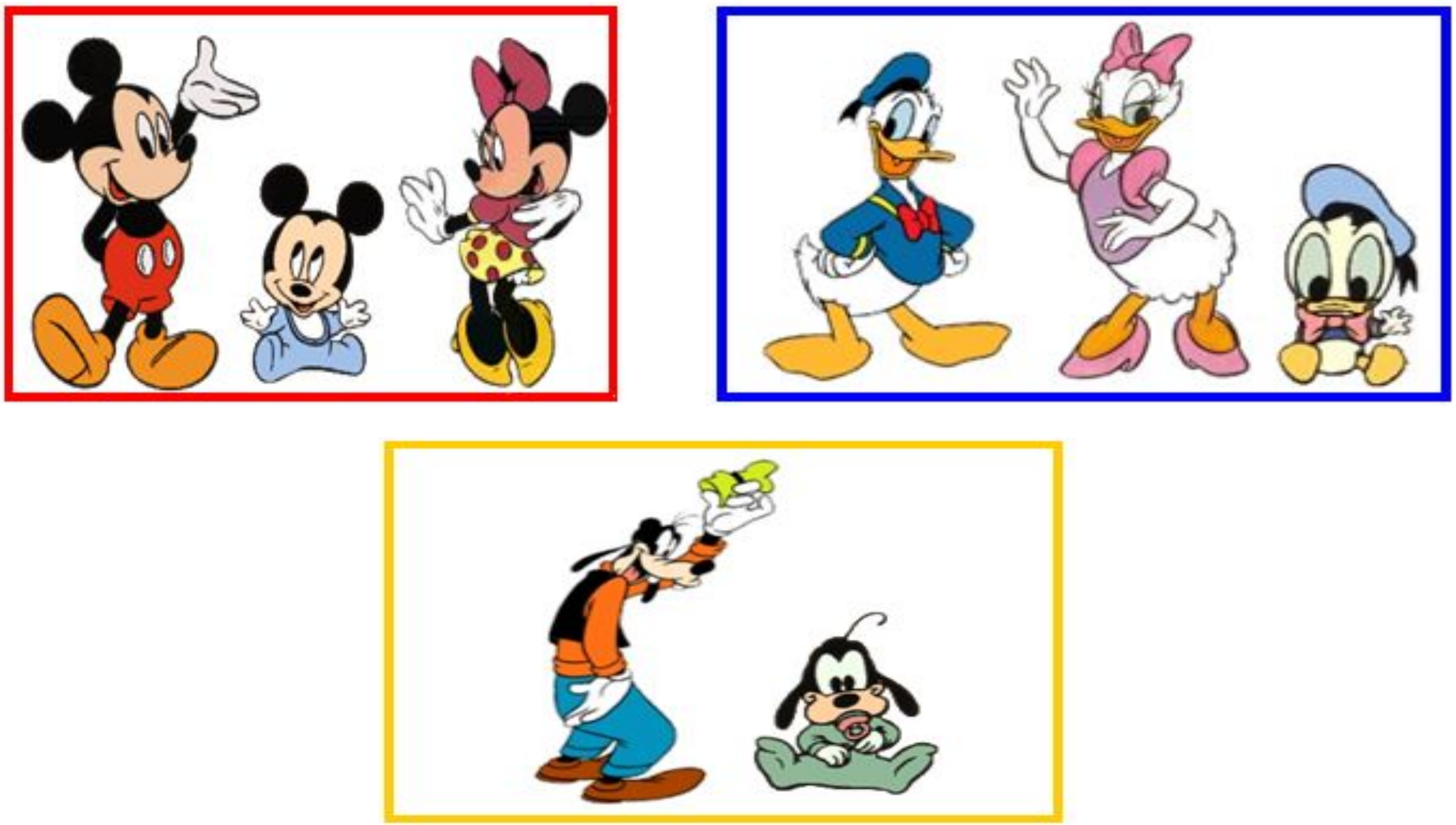
- Cluster: A collection of data objects
  - similar (or related) to one another within the same group
  - dissimilar (or unrelated) to the objects in other groups
- Cluster analysis (or *clustering*, *data segmentation, …*)
  - Finding similarities between data according to the characteristics found in the data and grouping similar data objects into clusters
- Unsupervised learning: no predefined classes (i.e., *learning by observations* vs. learning by examples: supervised)
- Typical applications
  - As a stand-alone tool to get insight into data distribution
  - As a preprocessing step for other algorithms

What is Cluster Analysis?



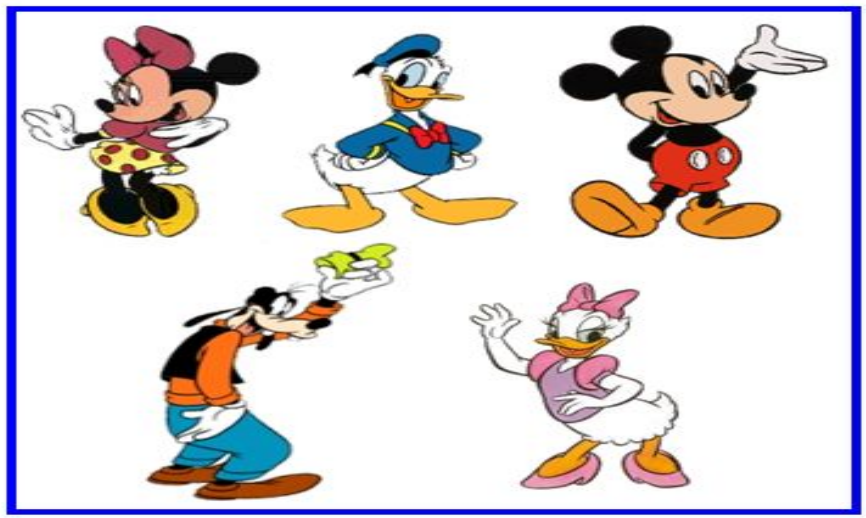
What is Cluster Analysis?

- By Family



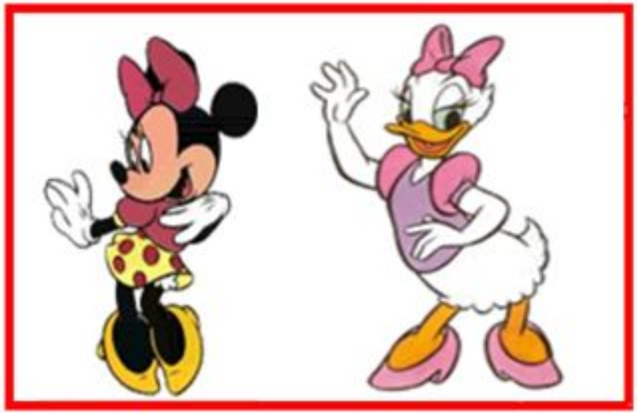
What is Cluster Analysis?

- By Age



What is Cluster Analysis?

- By Gender



Applications of Cluster Analysis

- Data reduction
  - Summarization: Preprocessing for regression, PCA, classification, and association analysis
  - Compression: Image processing: vector quantization
- Hypothesis generation and testing
- Prediction based on groups
  - Cluster & find characteristics/patterns for each group
- Finding K-nearest Neighbors
  - Localizing search to one or a small number of clusters
- Outlier detection: Outliers are often viewed as those “far away” from any cluster

Clustering: Application Examples

- Biology: taxonomy of living things: kingdom, phylum, class, order, family, genus and species
- Information retrieval: document clustering
- Land use: Identification of areas of similar land use in an earth observation database
- Marketing: Help marketers discover distinct groups in their customer bases, and then use this knowledge to develop targeted marketing programs
- City-planning: Identifying groups of houses according to their house type, value, and geographical location
- Earth-quake studies: Observed earth quake epicenters should be clustered along continent faults
- Climate: understanding earth climate, find patterns of atmospheric and ocean
- Economic Science: market research

Earthquakes

GLOBAL SEISMIC HAZARD MAP

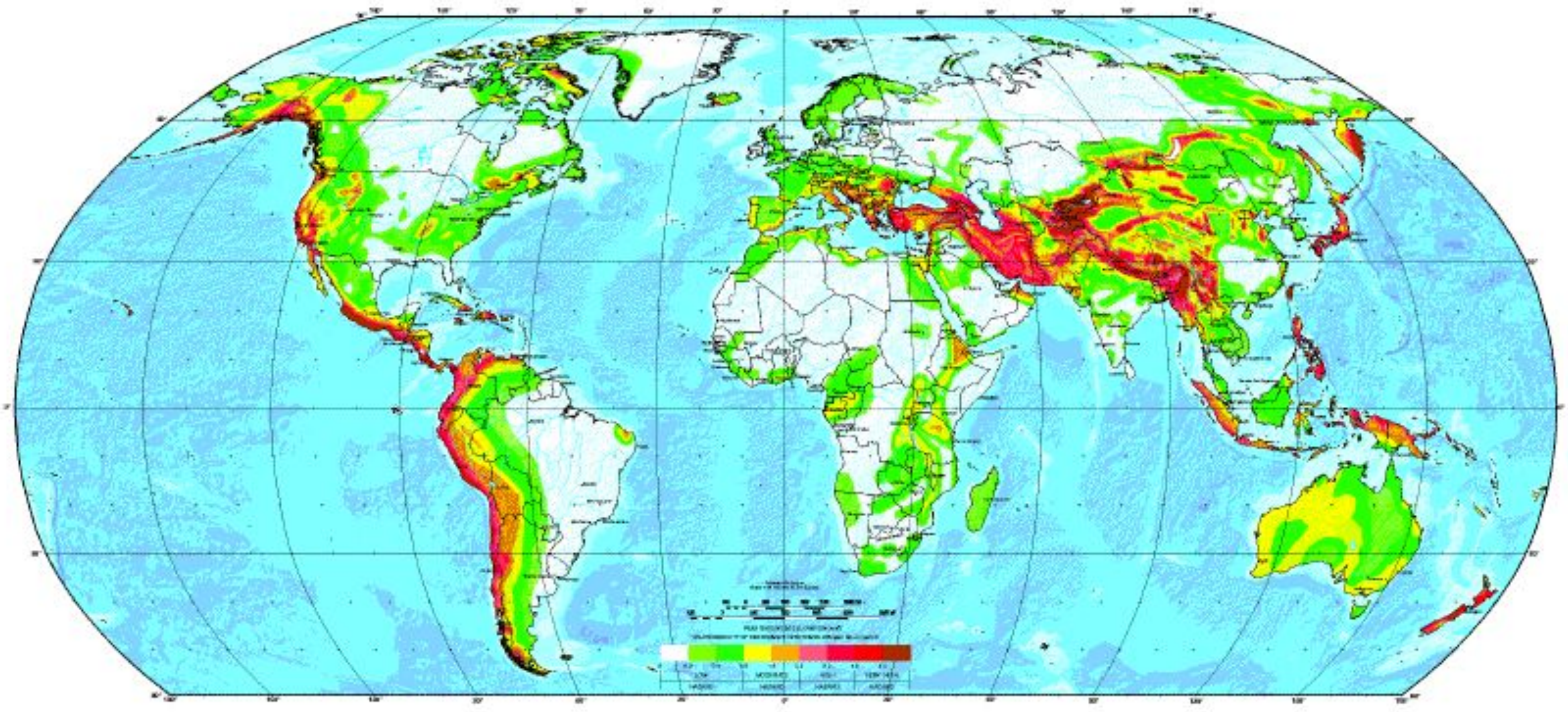


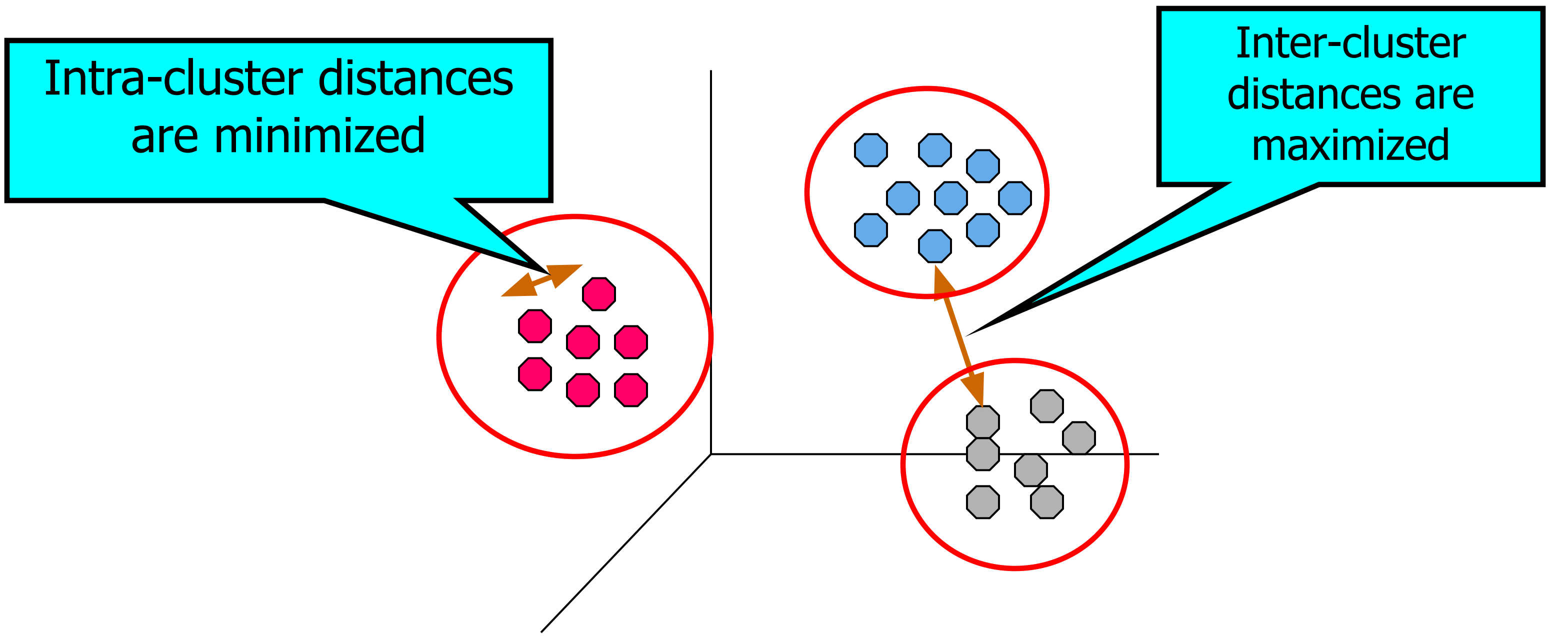
Image Segmentation

Basic Steps to Develop a Clustering Task

- Feature selection
  - Select info concerning the task of interest
  - Minimal information redundancy
- Proximity measure
  - Similarity of two feature vectors
- Clustering criterion
  - Expressed via a cost function or some rules
- Clustering algorithms
  - Choice of algorithms
- Validation of the results
  - Validation test (also, *clustering tendency* test)
- Interpretation of the results
  - Integration with applications

Quality: What Is Good Clustering?

- A good clustering method will produce high quality clusters
  - high intra-class similarity: cohesive within clusters
  - low inter-class similarity: distinctive between clusters
- The quality of a clustering method depends on
  - the similarity measure used by the method
  - its implementation, and
  - Its ability to discover some or all of the hidden patterns



Measure the Quality of Clustering

- Dissimilarity/Similarity metric
  - Similarity is expressed in terms of a distance function, typically metric: $d(i, j)$
  - The definitions of distance functions are usually rather different for interval-scaled, boolean, categorical, ordinal ratio, and vector variables
  - Weights should be associated with different variables based on applications and data semantics
- Quality of clustering:
  - There is usually a separate "quality" function that measures the "goodness" of a cluster.
  - It is hard to define "similar enough" or "good enough"
    - The answer is typically highly subjective

Considerations for Cluster Analysis

- Partitioning criteria
  - Single level vs. hierarchical partitioning (often, multi-level hierarchical partitioning is desirable)
- Separation of clusters
  - Exclusive (e.g., one customer belongs to only one region) vs. non-exclusive (e.g., one document may belong to more than one class)
- Similarity measure
  - Distance-based (e.g., Euclidian, road network, vector) vs. connectivity-based (e.g., density or contiguity)
- Clustering space
  - Full space (often when low dimensional) vs. subspaces (often in high-dimensional clustering)

Requirements and Challenges

- Scalability
  - Clustering all the data instead of only on samples
- Ability to deal with different types of attributes
  - Numerical, binary, categorical, ordinal, linked, and mixture of these
- Constraint-based clustering
  - User may give inputs on constraints
  - Use domain knowledge to determine input parameters
- Interpretability and usability
- Others
  - Discovery of clusters with arbitrary shape
  - Ability to deal with noisy data
  - Incremental clustering and insensitivity to input order
  - High dimensionality

Major Clustering Approaches 1

- Partitioning approach:
  - Construct various partitions and then evaluate them by some criterion, e.g., minimizing the sum of square errors
  - Typical methods: k-means, k-medoids, CLARANS
- Hierarchical approach:
  - Create a hierarchical decomposition of the set of data (or objects) using some criterion
  - Typical methods: Diana, Agnes, BIRCH, CHAMELEON
- Density-based approach:
  - Based on connectivity and density functions
  - Typical methods: DBSACN, OPTICS, DenClue
- Grid-based approach:
  - based on a multiple-level granularity structure
  - Typical methods: STING, WaveCluster, CLIQUE

Major Clustering Approaches 2

- Model-based:
  - A model is hypothesized for each of the clusters and tries to find the best fit of that model to each other
  - Typical methods: EM, SOM, COBWEB
- Frequent pattern-based:
  - Based on the analysis of frequent patterns
  - Typical methods: p-Cluster
- User-guided or constraint-based:
  - Clustering by considering user-specified or application-specific constraints
  - Typical methods: COD (obstacles), constrained clustering
- Link-based clustering:
  - Objects are often linked together in various ways
  - Massive links can be used to cluster objects: SimRank, LinkClus

Partitioning Algorithms: Basic Concept

- Partitioning method: Partitioning a database ***D*** of ***n*** objects into a set of ***k*** clusters, such that the sum of squared distances is minimized (where c_i is the centroid or medoid of cluster C_i)

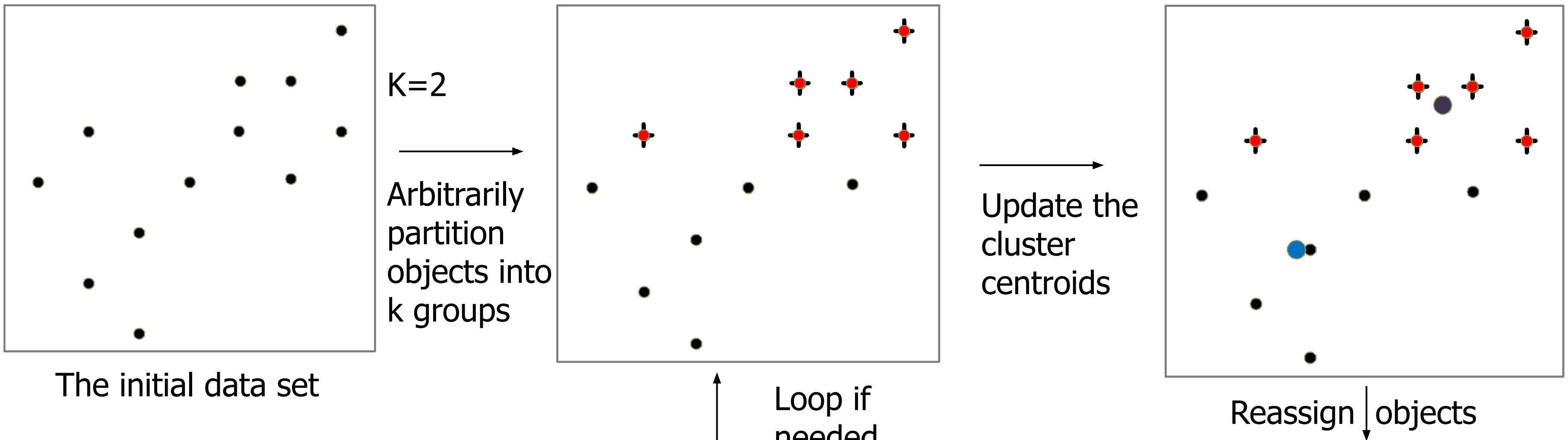
$$E = \Sigma_{i=1}^{k} \Sigma_{p \in C_i} (d(p, c_i))^2$$

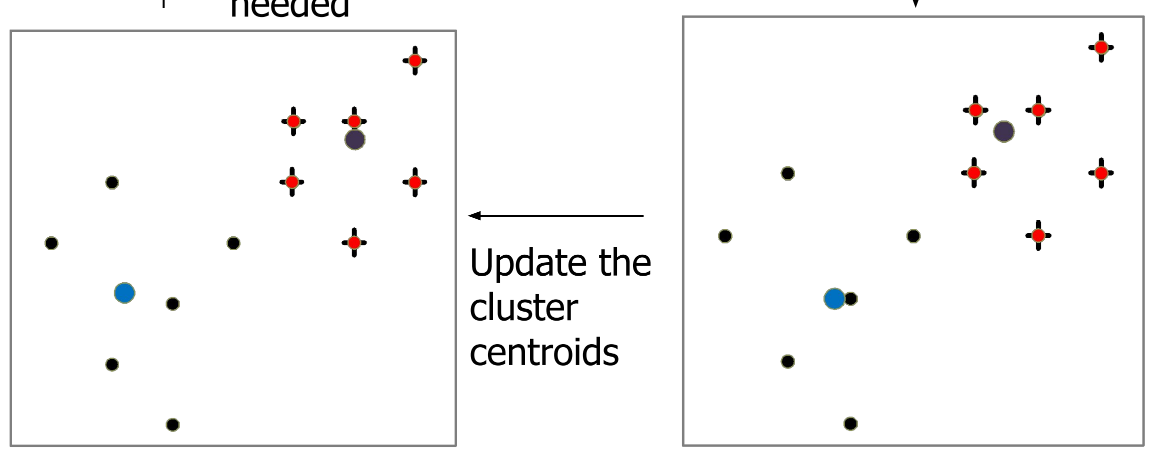
- Given *k*, find a partition of *k clusters* that optimizes the chosen partitioning criterion
  - Global optimal: exhaustively enumerate all partitions
  - Heuristic methods: *k-means* and *k-medoids* algorithms
  - *k-means* (MacQueen'67, Lloyd'57/'82): Each cluster is represented by the center of the cluster
  - *k-medoids* or PAM (Partition around medoids) (Kaufman & Rousseeuw'87): Each cluster is represented by one of the objects in the cluster

The *K-Means* Clustering Method

- Given *k*, the *k-means* algorithm is implemented in four steps:
  1. Partition objects into *k* nonempty subsets
  2. Compute seed points as the centroids of the clusters of the current partitioning (the centroid is the center, i.e., *mean point*, of the cluster)
  3. Assign each object to the cluster with the nearest seed point
  4. Go back to Step 2, stop when the assignment does not change

An Example of K-Means Clustering





- Partition objects into *k* nonempty subsets
- Repeat
  - Compute centroid (i.e., mean point) for each partition
  - Assign each object to the cluster of its nearest centroid
- Until no change

k-Means Algorithms steps

1. Selecting the desired number of clusters k
2. Initialization k cluster centers (centroid) random
3. Place any data or object to the nearest cluster. The proximity of the two objects is determined based on the distance. The distance used in k-means algorithm is the Euclidean distance (d)

$$d_{Euclidean}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

x = x1, x2,. , , , Xn and y = y1, y2,. , , , Yn n is the number of attributes (columns) between 2 records

4. Recalculate cluster centers with the current cluster membership. Cluster center is the average (mean) of all the data, or objects in a particular cluster
5. Assign each object again using the new cluster centers. If the cluster centers have not changed again, then the clustering process is completed. Or, go back to step 3 until the cluster centers do not change anymore (stable) or no significant decrease of the value of SSE (Sum of Squared Errors)

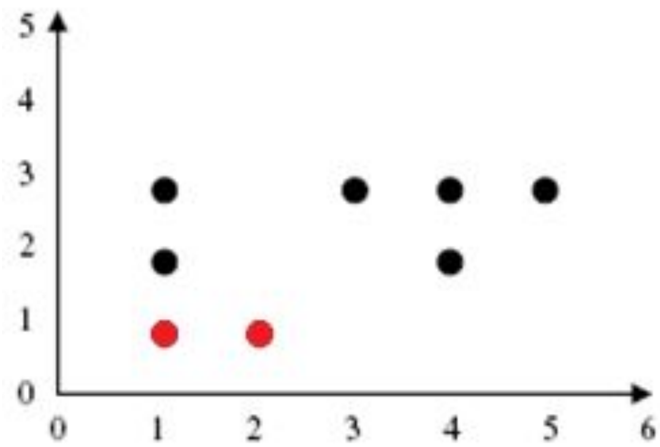
Exercise – Iteration 1

[STEP 1] Determine the number of clusters k = 2

[STEP 2] Determine the initial centroid random example of the data besides m1 = (1,1), m2 = (2,1)

[STEP 3] Place each object to the closest cluster centroid based on the closest value difference (distance).

			m1	m2	
Instances	x	y	(1,1)	(2,1)	Cluster Membership
A	1	3	2	3	m1
B	3	3	4	3	m2
C	4	3	5	4	m2
D	5	3	6	5	m2
E	1	2	1	2	m1
F	4	2	4	3	m2
G	1	1	0	1	m1
H	2	1	1	0	m2



Exercise – Iteration 1

[STEP 4] The results obtained, members cluster1 = {A, E, G}, cluster2 = {B, C, D, F, H}. The SSE(Sum of Square Error) and MSE(Mean Square Error) values are:

$$SSE = \sum_{i=1}^{k} \sum_{p \in C_i} d(p, m_i)^2$$

$$\begin{aligned} SSE = &\ \mathbf{[(1-1)^2+(3-1)^2] + [(1-1)^2+(2-1)^2] + [(1-1)^2+(1-1)^2]} \\ &+ [(3-2)^2+(3-1)^2] + [(4-2)^2+(3-1)^2] + [(5-2)^2+(3-1)^2] \\ &+ [(4-2)^2+(2-1)^2] + [(2-2)^2+(1-1)^2] \end{aligned}$$

$$SSE = \mathbf{4 + 1 + 0} + 5 + 8 + 13 + 5 + 0 = 36$$

$$MSE = 36/8 = 4.5$$

Cluster1	Cluster 2
(1, 3)	(3, 3)
(1, 2)	(4, 3)
(1, 1)	(5, 3)
	(4, 2)
	(2, 1)
--------	--------
(3/3, 6/3)	(18/5, 12/5)
(1, 2)	(3.6, 2.4)

[STEP 5] The new **centroid** for

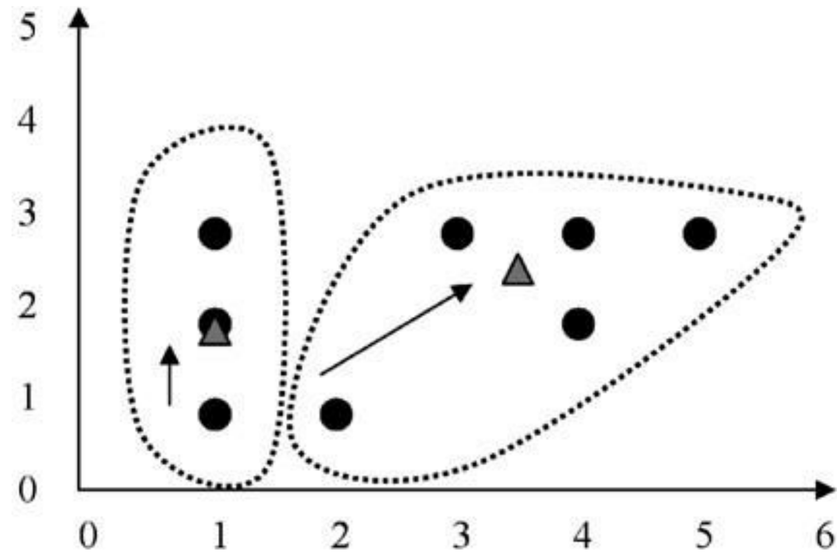
cluster1 = [(1 + 1 + 1)/3, (3 + 2 + 1)/3] = (1, 2) and

cluster2 = [(3 + 4 + 5 + 4 + 2)/5, (3 + 3 + 3 + 2 + 1)/5] = (3.6, 2.4)

Exercise – Iteration 2

[STEP 3] Place each object to the closest cluster centroid based on the closest value difference (distance).

			m1	m2	
Instances	x	y	(1,2)	(3.6,2.4)	Cluster Membership
A	1	3	1	3.2	m1
B	3	3	3	1.2	m2
C	4	3	4	1	m2
D	5	3	5	2	m2
E	1	2	0	3	m1
F	4	2	3	0.8	m2
G	1	1	1	4	m1
H	2	1	2	3	m1



Exercise – Iteration 2

[STEP 4] The results obtained, members cluster1 = {A, E, G, H}, cluster2 = {B, C, D, F}. The new SSE and MSE values are:

$$SSE = \sum_{i=1}^{k} \sum_{p \in C_i} d\left(p, m_i\right)^2$$

SSE = **[(1-1)^2+(3-2)^2]+ [(1-1)^2+(2-2)^2]+ [(1-1)^2+(1-2)^2] + [(2-1)^2+(1-2)^2]**
+ [(3-3.6)^2+(3-2.4)^2]+ [(4-3.6)^2+(3-2.4)^2]+ [(5-3.6)^2+(3-2.4)^2]
+ [(4-3.6)^2+(2-2.4)^2]

SSE = **1 + 0 + 1 + 2** + 0.72 + 0.52 + 2.32 + 0.32 = 7.88

MSE = 7.88/8 = 0.985

[STEP 5] The new centroid for

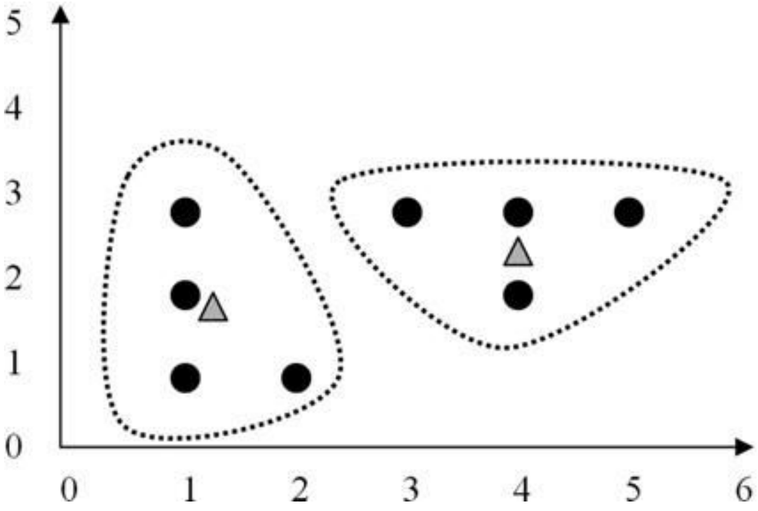
cluster1 = [(1 + 1 + 1+2)/4, (3 + 2 + 1+1)/4] = (1.25,1.75) and

cluster2 = [(3 + 4 + 5 + 4)/4, (3 + 3 + 3 + 2)/4] = (4, 2.75)

Exercise – Iteration 3

[STEP 3] Place each object to the closest cluster centroid based on the closest value difference (distance).

			m1	m2	
Instances	x	y	(1.25,1.75)	(4, 2.75)	Cluster Membership
A	1	3	1.5	3.25	m1
B	3	3	3	1.25	m2
C	4	3	4	0.25	m2
D	5	3	5	1.25	m2
E	1	2	0.5	3.75	m1
F	4	2	3	0.75	m2
G	1	1	1	4.75	m1
H	2	1	1.5	3.75	m1



Exercise – Iteration 3

[STEP 4] The results obtained, members cluster1 = {A, E, G, H}, cluster2 = {B, C, D, F }. The new SSE and MSE values are:

$$SSE = \sum_{i=1}^{k} \sum_{p \in C_i} d\left(p, m_i\right)^2$$

SSE = **[(1-1.25)^2+(3-1.75)^2]+ [(1-1.25)^2+(2-1.75)^2]+ [(1-1.25)^2+(1-1.75)^2] + [(2-1.25)^2+(1-1.75)^2]** + [(3-4)^2+(3-2.75)^2]+ [(4-4)^2+(3-2.75)^2]+ [(5-4)^2+(3-2.75)^2] + [(4-4)^2+(2-2.75)^2]

SSE = **1.625 + 0.125 + 0.625 + 0.125** + 1.0625 + 0.0625 + 1.0625 + 0.5625 = 7.88

MSE = 6.25/8 = 0.78125

[STEP 5] The new centroid for

cluster1 = [(1 + 1 + 1+2)/4, (3 + 2 + 1+1)/4] = (1.25,1.75) and

cluster2 = [(3 + 4 + 5 + 4)/4, (3 + 3 + 3 + 2)/4] = (4, 2.75)

Final Result

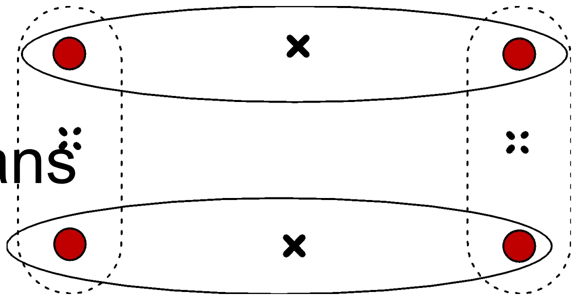
- Can be seen in the table. No change of members again on each cluster
  - The final results are:
  - cluster1 = {A, E, G, H}, and
  - cluster2 = {B, C, D, F}
  - With SSE value = 6.25 and MSE value = 0.78125
  - the number of iterations 3

Comments on the *K-Means* Method

- Strength:
  - *Efficient*: $O(tkn)$, where n is # objects, k is # clusters, and t is # iterations. Normally, k, t << n.
  - Comparing: PAM: $O(k(n-k)^2)$, CLARA: $O(ks^2 + k(n-k))$
- Comment: Often terminates at a *local optimal*
- Weakness
  - Applicable only to objects in a continuous n-dimensional space
    - Using the k-modes method for categorical data
    - In comparison, k-medoids can be applied to a wide range of data
  - Need to specify *k*, the *number* of clusters, in advance (there are ways to automatically determine the best *k* (see Hastie et al., 2009)
  - Sensitive to noisy data and *outliers*
  - Not suitable to discover clusters with *non-convex shapes*

Variations of the *K-Means* Method

- Most of the variants of the *k-means* which differ in
  - Selection of the initial *k* means
  - Dissimilarity calculations
  - Strategies to calculate cluster means



- Handling categorical data: *k-modes*
  - Replacing means of clusters with <u>modes</u>
  - Using new dissimilarity measures to deal with categorical objects
  - Using a <u>frequency</u>-based method to update modes of clusters
  - A mixture of categorical and numerical data: *k-prototype* method

What Is the Problem of the K-Means Method?

- The k-means algorithm is sensitive to outliers!
  - Since an object with an extremely large value may substantially distort the distribution of the data
- K-Medoids:
  - Instead of taking the **mean** value of the object in a cluster as a reference point, **medoids** can be used, which is the **most centrally located** object in a cluster